Rethinking Pruning Large Language Models: Benefits and Pitfalls of Reconstruction Error Minimization

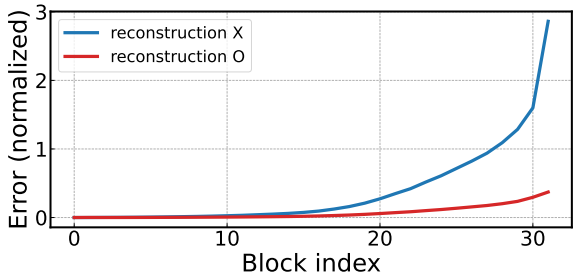
Sungbin Shin[1*] **Wonpyo Park**[2] **Jaeho Lee**[1,2,3] **Namhoon Lee**[1,2,3]
[1]POSTECH [2]Google [3]Yonsei University
{ssbin4,jaeho.lee,namhoonlee}@postech.ac.kr
wppark@google.com

Abstract

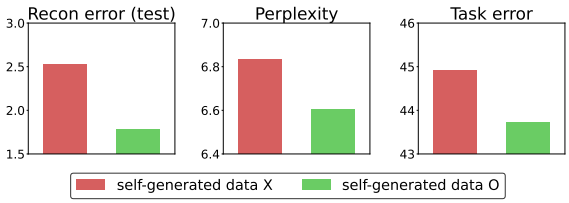
This work suggests fundamentally rethinking the current practice of pruning large language models (LLMs). The way it is done is by divide and conquer: split the model into submodels, sequentially prune them, and reconstruct predictions of the dense counterparts on small calibration data one at a time; the final model is obtained simply by putting the resulting sparse submodels together. While this approach enables pruning under memory constraints, it generates high reconstruction errors. In this work, we first present an array of reconstruction techniques that can significantly reduce this error by more than 90%. Unwittingly, however, we discover that minimizing reconstruction error is not always ideal and can overfit the given calibration data, resulting in rather increased language perplexity and poor performance at downstream tasks. We find out that a strategy of self-generating calibration data can mitigate this trade-off between reconstruction and generalization, suggesting new directions in the presence of both benefits and pitfalls of reconstruction for pruning LLMs.[1]

1 Overview

Large language models (LLMs) have shown remarkable potential and achieved tremendous successes in various domains (Brown et al., 2020; Singhal et al., 2023; Roziere et al., 2023). Nevertheless, running them requires a significant amount of computations and memory, raising concerns about accessibility, sustainability, and scalability (Strubell et al., 2019; Bender et al., 2021). Neural network pruning holds great promise for mitigating this issue (LeCun et al., 1989; Hoefler et al., 2021). A complication here is that the standard approach is not quite feasible since it usually involves an extensive training process (and training data) which is challenging to carry out for LLMs.



(a) Effects of reconstruction techniques on reducing the error



(b) Effects of self-generated data on mitigating overfitting

Figure 1: (a) Reconstruction techniques significantly reduce the compounding errors and lead to a substantial reduction of error in the final block. Reconstruction O and X refer to the results with and without the proposed reconstruction techniques (BR, GP, CR) respectively. (b) Minimizing reconstruction error may not always be ideal since models can overfit calibration data (we show this in Section 3.2). Using our self-generated calibration data in the reconstruction process mitigates this issue quite effectively by decreasing test error, perplexity, and error rates for downstream tasks.

To address this issue, LLM pruning is done post training. Specifically, it could be formulated as a reconstruction problem as follows:

$$\begin{aligned} \min_{w,m} \quad & \|f(\bar{w};\mathcal{D}) - f(m \odot w;\mathcal{D})\|_2^2 \\ \text{s.t.} \quad & \|m\|_0 \leqslant k \,, \end{aligned} \tag{1}$$

i.e., given a pre-trained model $\bar{w}$, the goal is to find a pruning mask m such that the resulting sparse model $m \odot w$ reconstructs the predictions of the

[*]Work partly done as a student researcher at Google
[1]Our code is available at https://github.com/LOG-postech/rethinking-LLM-pruning.

Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 1182–1191
November 12-16, 2024 ©2024 Association for Computational Linguistics

original dense model $f(\bar{w}; \cdot)$ on some calibration data $\mathcal{D}$; here, $\odot$ denotes element-wise product for vectorized representations, and m needs to satisfy a given sparsity constraint k. If the objective criterion—*reconstruction error*—is minimized to zero, then we achieve the perfect reconstruction and thereby pruning results.

While one could now avoid training LLMs from scratch with (1), it still requires as much memory as of the given LLM, hindering development under memory constraints. To circumvent this issue, many recent works take a divide-and-conquer approach: *i.e.*, split the model into a sequence of smaller submodels, prune and reconstruct each submodel individually, and simply put all resulting sparse submodels together (Frantar and Alistarh, 2023; Sun et al., 2024; Zhang et al., 2024). Albeit fairly effective, we find that this can easily create critically high compounding errors. This is because solutions for each subproblem yield non-zero reconstruction errors.

In this work, we address the reconstruction error minimization for pruning LLMs with the following three major pillars. First, we focus on developing various engineering techniques to reduce this error. These are inspired to lessen the suboptimality of subsolutions by incorporating different levels of extension schemes. Second, we suggest that reducing this error is not necessarily favorable, however. Our extensive experimental results indicate that it is possibly due to overfitting, given limited calibration data and high problem complexity. Third, we present useful strategies to potentially mitigate the risk of reconstruction and improve generalization. This is based on what we call the self-generation of calibration data.

Briefly, this work investigates the benefits and pitfalls of the reconstruction error minimization scheme for pruning LLMs. To our best knowledge, this trade-off has not been explicitly identified or studied before, thereby suggesting rethinking the current practice. Our initial investigations may shed light on some potential future research directions. We summarize our main results in Figure 1.

2 Reconstruction Techniques

This section explains three optimization schemes we use to reduce reconstruction errors in this work.

Block-wise reconstruction (BR) The seminal work of Frantar and Alistarh (2023) proposes to reconstruct predictions layer-wise based on least squares. By removing non-linearity this approach yields a closed-form solution. However, we find that this can create a high reconstruction error since the system is highly underdetermined (*i.e.*, there are much more parameters than calibration data). To reduce compounding errors, we first consider extending the unit of optimization target from a layer to a block of layers. Specifically, this means a block-wise reconstruction (BR) which can be formulated as follows:

$$\min_{w_1,\ldots,w_B} \sum_{i=1}^{B} \|g_i(\bar{w}_i; x_i) - g_i(\bar{m}_i \odot w_i; x_i)\|_2^2 \quad (2)$$

where g_i refers to the i-th block of layers (*e.g.*, a Transformer block) in which we have the optimization variables w_i, and x_i denotes the inputs to the i-th block which originally come from calibration data; here, the pruning mask $\bar{m}$ is fixed assuming that it is already obtained from an arbitrary pruning method. *I.e.*, the goal is to update variables in each block to minimize the extended reconstruction errors. We solve this problem iteratively using the standard gradient-based method. Notably a similar approach is also proposed in the concurrent work of Guo et al. (2024), and we find in our experiments that BR is extremely effective in reducing the reconstruction errors in Section 3.1. We illustrate the idea of BR in Figure 2.

Global propagation (GP) While the general divide-and-conquer principle is quite functional, we identify a potential issue therein: by sequentially solving the subproblem, it is constantly fitting practically suboptimal solutions obtained from the previous step (which become gradually worse), as with $x_i = g_{i-1}(\bar{m}_{i-1} \odot w_{i-1}; x_{i-1})$. We realize that this is another source of compounding errors, and thus, suggest that when we locally reconstruct a model, at least we use global propagation (GP) from the original dense model as input to the target reconstruction; *i.e.*, $x_i = g_{i-1}(\bar{w}_{i-1}; x_{i-1})$. We show that GP improves the reconstruction results quite significantly in Section 3.1. We further note that a similar principle is found in various applications including low-rank approximation (Zhang et al., 2015), channel pruning (He et al., 2017), and quantization (Nagel et al., 2020; Hubara et al., 2021). We illustrate the idea of GP in Figure 2.

Cross-block reconstruction (CR) Another way we consider to further reduce reconstruction errors is to extend the reconstruction unit from a block to

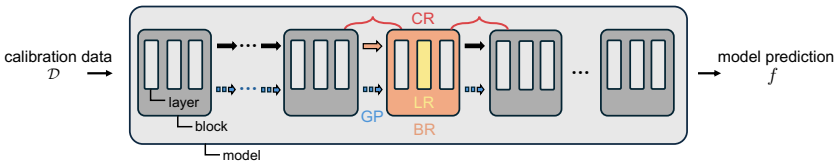


Figure 2: An illustration of reconstruction techniques for pruning large language models. Here, we want the sparse model $f(m \odot w; \cdot)$ to reconstruct the prediction of the dense model on some calibration data $\mathcal{D}$. LR, BR, GP, and CR each correspond to layer-wise reconstruction, block-wise reconstruction, global propagation, and cross-block reconstruction. Here, solid and dashed arrows each represent the inputs coming from sparse and dense models.

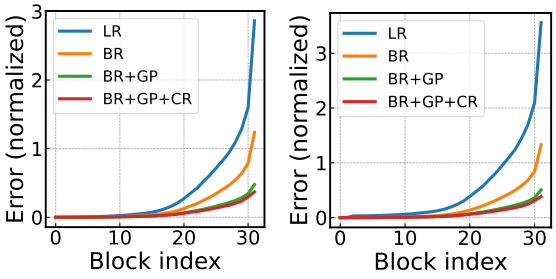
multiple blocks and stitch the solutions in between by connecting via the adjacent blocks. Specifically, this means that now g in (2) becomes a composite of multiple blocks, say h, and we ensure h overlaps; more precisely, $h_i = g_i \circ g_{i-1}$ and $h_{i+1} = g_{i+1} \circ g_i$ for two blocks, and so on for all blocks. This way, namely cross-block reconstruction or CR (Ding et al., 2023), we can potentially bridge between subsolutions by taking into account some interaction between adjacent blocks, and hence, reduce the compounding errors. We illustrate the idea of CR in Figure 2.

To elaborate further, the difference between BR and CR is that while BR is about updating parameters within a block (thus it is not concerned with how to combine subsolutions), CR takes a step further and is about stitching the subsolutions; *i.e.*, CR updates parameters within two adjacent blocks, and when it comes to reconstructing the next block, it includes the overlapping block so that it has the effect of "stitching". This method is found to be quite effective for reducing the error, however, we find that this method can often lead to overfitting. We discuss this in detail in Section 3.2.

3 Experiments

3.1 Reconstruction error

We first evaluate the effectiveness of the suggested techniques in reducing the reconstruction error. Here, we focus on pruning LLaMA-7B (Touvron et al., 2023) and OPT-125M (Zhang et al., 2022) to unstructured 50% sparsity with three pruning methods: SparseGPT (Frantar and Alistarh, 2023), Wanda (Sun et al., 2024), and Magnitude (Han et al., 2015). For each pruning method, we examine four reconstruction strategies: layer-wise reconstruction (LR), block-wise reconstruction (BR), block-wise reconstruction with global propagation (BR+GP), and cross-block reconstruction with global propagation (BR+GP+CR). Following the convention, we use 256 calibration data randomly



(a) SparseGPT (b) Wanda

Figure 3: Results of reconstruction techniques for LLaMA-7B. They constantly reduce the compounding errors, achieving a significant decrease at the final block ($\sim$ 90%). We find this trend is consistent across different settings. See Figures 5 and 6 of Appendix B for more results.

sampled from C4 (Raffel et al., 2020) each containing 1024 tokens. We run the Adam optimizer for 10 epochs (see Appendix A for details). The results are presented in Figure 3.

We can see that all the reconstruction techniques reduce the compounding errors quite significantly, yielding a substantial reduction at the final block. Specifically, BR first reduces the final error by at least 50% across all pruning methods compared to LR, BR+GP further reduces the error by at least 60% compared to BR, and finally, BR+GP+CR reduces the error by at least 20% compared to BR+GP. Consequently, we observe that the error is reduced from 87% to 94% with BR+GP+CR compared to the baseline LR.

3.2 Generalization performance

We now evaluate the generalization performances of the reconstruction results. Specifically, we measure the perplexity of the pruned model on three different datasets: raw-Wikitext2 (Merity et al., 2017), PTB (Marcus et al., 1994), and validation data of C4. We also measure its zero-shot task performance in accuracy on seven downstream tasks: BoolQ (Clark et al., 2019), RTE (Wang et al., 2019),

Pruner	Reconstruction	Error (normalized)	Perplexity				Zero-shot accuracy							
			Wiki	PTB	C4	Mean	BoolQ	RTE	HellaSwag	WinoGrande	ARC-e	ARC-c	OpenbookQA	Mean
Dense	–	–	5.68	10.12	7.34	7.71	75.11	66.43	56.96	70.00	75.29	41.81	34.40	60.00
SparseGPT	LR	2.86	7.24	12.61	9.17	9.67	73.36	58.12	51.86	68.90	70.62	36.95	28.60	**55.49**
	BR	1.24	6.82	11.69	8.66	9.06	71.71	54.51	52.54	68.27	71.68	36.18	28.40	54.76
	BR+GP	0.48	6.72	11.32	8.55	**8.86**	71.22	53.79	53.57	68.90	71.76	37.54	27.80	54.94
	BR+GP+CR	**0.37**	6.83	11.41	8.71	8.99	72.91	55.60	53.24	68.51	71.21	36.26	27.80	55.07
Wanda	LR	3.56	7.25	12.77	9.28	9.77	71.28	55.23	52.04	66.46	69.36	36.52	28.80	54.24
	BR	1.33	6.82	11.54	8.70	9.02	72.02	57.04	52.45	67.09	72.18	36.60	28.60	55.14
	BR+GP	0.51	6.68	11.25	8.56	**8.83**	72.66	60.29	53.25	68.43	71.46	37.63	29.80	**56.22**
	BR+GP+CR	**0.38**	6.79	12.01	8.72	9.18	73.00	59.93	53.18	68.27	71.13	37.29	28.80	55.94
Magnitude	LR	8.08	17.29	49.67	23.78	30.25	54.65	54.15	45.47	59.43	58.75	33.45	22.60	46.93
	BR	2.37	7.83	15.73	9.66	11.07	68.90	49.82	47.85	66.38	70.29	36.77	27.00	52.43
	BR+GP	0.63	6.88	11.77	8.77	**9.14**	71.65	52.35	53.00	68.19	70.75	37.63	29.00	**54.65**
	BR+GP+CR	**0.46**	6.98	11.96	8.85	9.27	72.23	48.74	53.20	67.09	70.54	36.95	28.20	53.85

Table 1: Effects of different reconstruction techniques on error, perplexity, and zero-shot accuracy for LLaMA-7B. **Bold** and underline refer to best in general and task-specific. See Table 3 of Appendix B for the OPT-125M results.

Pruner	CR	Error (normalized)	
		Calib	Test
SparseGPT	X	0.006	0.0083
	O	**0.004**	**0.0078**
Wanda	X	0.006	0.0080
	O	**0.004**	**0.0076**
Magnitude	X	0.008	0.0109
	O	**0.005**	**0.0102**

(a) OPT-125M

Pruner	CR	Error (normalized)	
		Calib	Test
SparseGPT	X	0.48	**2.30**
	O	**0.37**	2.53
Wanda	X	0.51	**2.23**
	O	**0.38**	2.48
Magnitude	X	0.63	**2.42**
	O	**0.46**	2.55

(b) LLaMA-7B

Table 2: Reconstruction errors of OPT-125M and LLaMA-7B on test data (raw-Wikitext2) as well as calibration data. Overfitting by CR is only observed for the larger LLaMA-7B model. We find that larger models in general are more susceptible to overfitting. See Tables 3 and 4 of Appendix B for more results.

HellaSwag (Zellers et al., 2019), Winogrande (Sakaguchi et al., 2020), ARC Easy and Challenge (Clark et al., 2018), and OpenbookQA (Mihaylov et al., 2018). The results are presented in Table 1.

At first, we find that the perplexity effectively decreases with BR and GP; the value reduces across all test cases including different models, pruning methods, and datasets. Unexpectedly, however, the perplexity rather increases when we add CR despite the reduced reconstruction errors. We also observe a similar trend in zero-shot performance for Wanda and Magnitude pruning, with mean accuracy increasing by a large margin with BR and GP but decreasing with CR. Interestingly, for SparseGPT, reconstruction techniques do not generally help zero-shot performance. We hypothesize that it is because SparseGPT already conducts fairly heavy optimization compared to other methods, and applying further reconstruction on particular calibration data may not help improve zero-shot performance since it is more sensitive to distribution shift. Furthermore, we find that such overfiting tends to occur more for LLaMA-7B than OPT-125M (see Table 2). This is possibly due to model size; *i.e.*, given the same amount of (limited) calibration data, over-optimizing can make large models more likely to overfit and lead to poor generalization.

We can summarize our findings are as follows.

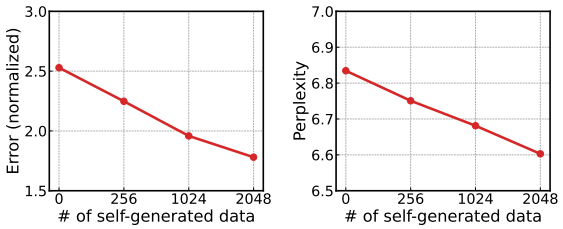
- BR and GP are found to be very effective in reducing perplexity in all cases; on the other hand, CR often leads to overfitting, especially for large models.
- This holds true for zero-shot performance as well, with only exception of SparseGPT, for which BR and GP do not help much in improving zero-shot performance; this is possibly due to the fact that SparseGPT already conducted fairly heavy optimization of remaining weights. It is also possible that adapting to downstream task is more prone to overfitting. This certainly requires more investigations.

In short, we can attempt to say without much loss of generality that "BR and GP can generally help for pruning LLMs in terms of reducing perplexity".

4 Further Exploration

We have seen that reconstruction techniques are useful but they can lead to undesirable overfitting. Here we explore potential ways to alleviate this risk. In particular, we identify that the calibration data is highly limited in two aspects: it is too little (compared to optimization variables)[2] and does not represent the training data (as it is arbitrarily given); the former is related to the general representation-generalization complexity trade-off, and the latter is about whether the reconstruction can mimic the behavior of the original model.

[2]This can be especially problematic for domain-specific LLMs, *e.g.*, healthcare (Singhal et al., 2023; Luo et al., 2022) and finance (Wu et al., 2023; Yang et al., 2023), where obtaining real-world data can be highly challenging due to privacy concerns.



(a) Test error (b) Perplexity

Figure 4: Effects of self-generated calibration data on (a) reconstruction error for test data (raw-Wikitext2) and (b) perplexity for LLaMA-7B; they both improve with more self-generation. See Figure 7 of Appendix B for more results.

To this end, we reflect on the fact that what we are dealing with is a *generative* (language) model, meaning that we can create calibration data that is potentially much bigger in size and closer to the original distribution. We find that this self-generation technique has recently been proposed in other contexts (Meng et al., 2022; Ye et al., 2022; Liu et al., 2023; Li et al., 2024), and thus, follow the process therein to produce high-quality text data. Using that, we perform reconstruction again, and the results are reported in Figure 4. We observe that making use of more self-generated calibration data (without unfairly violating the given setting) reduces both test error and perplexity, mitigating overfitting quite effectively.

5 Conclusion

In this work, we take a close look at the current practice of minimizing reconstruction errors for pruning LLMs. We first find that with various reconstruction techniques, one can reduce the error quite significantly and improve quality of pruning results on both language perplexity and zero-shot accuracy. Nevertheless, it turns out that decreasing error as it is now is not always desirable since it may cause overfitting calibration data. We present initial results that this issue can be potentially mitigated by self-generating calibration data. There are many remaining possibilities, and we believe our findings suggest opportunities for future work.

6 Limitations

There remain several limitations in our experiments and we plan to address these in future work. First, our main experiments are limited to LLaMA-7B and OPT-125M. We intend to scale up our experiments to much larger models of up to 70B parameters and different architectures including Mixtral (Jiang et al., 2024) or Gemma (Team et al., 2024). Next, reconstruction techniques BR, GP, and CR require additional memory compared to LR, although they still use much less memory compared to model-level reconstruction of solving (1) (see Appendix B for the details). We plan to introduce parameter-efficient optimization (Hu et al., 2022) to alleviate this increased memory burden.

Although the self-generation of calibration data effectively mitigates overfitting, it requires more computation for reconstruction. Finally, we find that some portions of the generated texts are far from plain English texts and thus may not serve as good calibration data (see Table 5 of Appendix C for the examples). In this regard, we believe that reducing the number of these irrelevant examples and generating only a few number of high-quality texts can be a potential way to improve performance and increase efficiency.

Acknowledgements

This work was partly supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2019-II191906, Artificial Intelligence Graduate School Program (POSTECH); RS-2022-II220959/No.2022-0-00959, (part2) Few-Shot learning of Causal Inference in Vision and Language for Decision Making; RS-2024-00338140, Development of Learning and Utilization Technology to Reflect Sustainability of Generative Language Models and Up-to-Dateness over Time) and the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (RS-2023-00210466, RS-2023-00265444, RS2023-0021371). Sungbin Shin was supported by Kwanjeong Educational Foundation Scholarship.

References

Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big? *FAccT*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *NeurIPS*.

Christopher Clark, Kenton Lee, Ming-Wei Chang,

Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. *NAACL*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Xin Ding, Xiaoyu Liu, Yun Zhang, Zhijun Tu, Wei Li, Jie Hu, Hanting Chen, Yehui Tang, Zhiwei Xiong, Baoqun Yin, et al. 2023. Cbq: Cross-block quantization for large language models. *arXiv preprint arXiv:2312.07950*.

Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive language models can be accurately pruned in one-shot. *ICML*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.

Song Guo, Fan Wu, Lei Zhang, Xiawu Zheng, Shengchuan Zhang, Fei Chao, Yiyu Shi, and Rongrong Ji. 2024. Ebft: Effective and block-wise fine-tuning for sparse llms. *arXiv preprint arXiv:2402.12419*.

Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. *NeurIPS*.

Yihui He, Xiangyu Zhang, and Jian Sun. 2017. Channel pruning for accelerating very deep neural networks. *ICCV*.

Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *JMLR*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. *ICLR*.

Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. 2021. Accurate post training quantization with small calibration sets. *ICML*.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Yann LeCun, John Denker, and Sara Solla. 1989. Optimal brain damage. *NeurIPS*.

Liang Li, Qingyuan Li, Bo Zhang, and Xiangxiang Chu. 2024. Norm tweaking: High-performance low-bit quantization of large language models. *AAAI*.

Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*.

Renqian Luo, Liai Sun, Yingce Xia, Tao Qin, Sheng Zhang, Hoifung Poon, and Tie-Yan Liu. 2022. Biogpt: generative pre-trained transformer for biomedical text generation and mining. *Briefings in bioinformatics*.

Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D Lee, Danqi Chen, and Sanjeev Arora. 2023. Fine-tuning language models with just forward passes. *NeurIPS*.

Mitch Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. 1994. The penn treebank: Annotating predicate argument structure. *HLT*.

Yu Meng, Jiaxin Huang, Yu Zhang, and Jiawei Han. 2022. Generating training data with language models: Towards zero-shot language understanding. *NeurIPS*.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. *ICLR*.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. *EMNLP*.

Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or down? adaptive rounding for post-training quantization. *ICML*.

Satya Sai Srinath Namburi, Makesh Sreedhar, Srinath Srinivasan, and Frederic Sala. 2023. The cost of compression: Investigating the impact of compression on parametric knowledge in language models. *EMNLP 2023 Findings*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. Winogrande: An adversarial winograd schema challenge at scale. *AAAI*.

Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. 2023. Large language models encode clinical knowledge. *Nature*.

Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in nlp. *ACL*.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2024. A simple and effective pruning approach for large language models. *ICLR*.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2019. Glue: A multi-task benchmark and analysis platform for natural language understanding. *ICLR*.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*.

Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*.

Jiacheng Ye, Jiahui Gao, Qintong Li, Hang Xu, Jiangtao Feng, Zhiyong Wu, Tao Yu, and Lingpeng Kong. 2022. Zerogen: Efficient zero-shot learning via dataset generation. *EMNLP*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *ACL*.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

Xiangyu Zhang, Jianhua Zou, Kaiming He, and Jian Sun. 2015. Accelerating very deep convolutional networks for classification and detection. *TPAMI*.

Yuxin Zhang, Lirui Zhao, Mingbao Lin, Yunyun Sun, Yiwu Yao, Xingjia Han, Jared Tanner, Shiwei Liu, and Rongrong Ji. 2024. Dynamic sparse no training: Training-free fine-tuning for sparse llms. *ICLR*.

A Experimental Details

Experiment configurations We run our experiments with a single A100 GPU having 80GB of memory. For BR and CR, we run the Adam optimizer for 10 epochs with a batch size of 8, without weight decay or gradient clipping. The learning rate is set to 0.0002 and decays linearly following Guo et al. (2024). For evaluating the performance on downstream tasks, we use the EleutherAI-evalharness framework (Gao et al., 2023).

Calculation of normalized reconstruction error The reconstruction error for i-th block is calculated as $\frac{1}{NHT}\|g_i(\bar{w}_i;\bar{x}_i)-g_i(m_i\odot w_i;x_i)\|_2^2$ where N, H, T each represent the number of calibration data, hidden dimension, and the token length. $\bar{x}_i, x_i$ represent the inputs coming from dense and sparse blocks respectively.

Licenses and uses of models and datasets LLaMA (Touvron et al., 2023) and OPT (Zhang et al., 2022) are released under non-commercial bespoke licenses. raw-Wikitext2 (Merity et al., 2017), PTB (Marcus et al., 1994), and C4 (Raffel et al., 2020) are released under CC BY-SA 4.0, LDC user agreement, and ODC-By. BoolQ (Clark et al., 2019), RTE (Wang et al., 2019), HellaSwag (Zellers et al., 2019), Winogrande (Sakaguchi et al., 2020), ARC (Clark et al., 2018), and OpeenbookQA (Mihaylov et al., 2018) are released under CC BY-SA 3.0, Apache 2.0, MIT License, Apache 2.0, CC BY-SA 4.0, and Apache 2.0 respectively. We confirm that these models and datasets are used for their intended use and the data does not contain personal information. EleutherAI-evalharness framework is released under the MIT License.

B Additional Results

More results on the reconstruction techniques Effects of reconstruction techniques on reducing the error for LLaMA-7B and OPT-125M are presented in Figures 5 and 6 respectively. It is clearly observed that different reconstruction techniques significantly reduce the error for all cases.

Effects of reconstruction techniques on performance for OPT-125M are presented in Table 3.

Pruner	Reconstruction	Error (normalized)	Perplexity				Zero-shot accuracy							
			Wiki	PTB	C4	Mean	BoolQ	RTE	HellaSwag	WinoGrande	ARC-e	ARC-c	OpenbookQA	Mean
Dense	–	–	27.66	38.99	26.56	31.07	55.44	50.18	29.19	50.20	43.60	19.03	16.6	37.75
SparseGPT	LR	0.019	36.35	54.93	33.12	41.47	61.31	48.01	28.29	53.28	40.19	19.28	15.60	**38.00**
	BR	0.008	31.94	45.75	29.91	35.87	60.49	47.65	28.44	51.38	42.17	19.88	14.60	37.80
	BR+GP	0.006	31.57	45.52	29.81	35.63	60.18	45.13	28.53	52.17	42.63	19.62	14.80	37.58
	BR+GP+CR	**0.004**	30.86	44.61	29.45	**34.97**	60.31	46.21	28.64	51.07	42.63	19.71	15.80	37.77
Wanda	LR	0.032	39.00	56.27	34.62	43.30	62.05	48.38	28.31	52.01	39.56	19.62	14.20	37.73
	BR	0.008	31.55	46.17	29.89	35.87	60.24	47.65	28.34	50.20	41.50	19.54	15.00	37.50
	BR+GP	0.006	31.18	45.47	29.67	35.44	59.85	48.01	28.66	51.54	41.71	19.28	16.20	**37.89**
	BR+GP+CR	**0.004**	30.59	44.80	29.33	**34.91**	58.81	45.85	28.68	50.99	42.34	19.03	15.00	37.24
Magnitude	LR	0.121	193.36	276.15	141.01	203.5	60.55	53.43	27.32	52.57	33.04	19.97	14.20	37.30
	BR	0.010	36.06	49.15	31.63	38.95	58.99	48.38	28.35	51.22	41.20	19.88	15.80	37.69
	BR+GP	0.008	35.56	48.17	31.75	38.50	58.20	49.46	28.44	51.54	42.26	19.88	15.20	**37.85**
	BR+GP+CR	**0.005**	33.76	46.84	30.88	**37.16**	57.28	45.49	28.53	51.93	42.00	19.97	15.60	37.26

Table 3: Effects of different reconstruction techniques on error, perplexity, and zero-shot accuracy for OPT-125M. **Bold** and underline refer to best in general and task-specific.

Pruner	CR	Error (normalized)			
		Calib	Test (Wiki)	Test (PTB)	Tets (C4)
SparseGPT	X	0.006	0.0083	0.009	0.0065
	O	**0.004**	**0.0078**	**0.0083**	**0.0061**
Wanda	X	0.006	0.008	0.0088	0.0061
	O	**0.004**	**0.0076**	**0.0082**	**0.0058**
Magnitude	X	0.008	0.0109	0.0115	0.0125
	O	**0.005**	**0.0102**	**0.0111**	**0.0099**

(a) OPT-125M

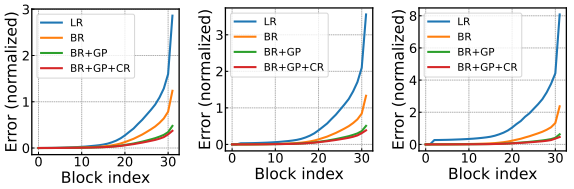
Pruner	CR	Error (normalized)			
		Calib	Test (Wiki)	Test (PTB)	Tets (C4)
SparseGPT	X	0.48	**2.30**	**2.29**	**1.99**
	O	**0.37**	2.53	2.60	2.31
Wanda	X	0.51	**2.23**	**2.29**	**1.98**
	O	**0.38**	2.48	2.86	2.31
Magnitude	X	0.63	**2.42**	**2.72**	**2.21**
	O	**0.46**	2.55	3.03	2.40

(b) LLaMA-7B

Table 4: Reconstruction errors of OPT-125M and LLaMA-7B on test data (raw-Wikitext2) as well as calibration data. Overfitting by CR is only observed for the larger LLaMA-7B model.

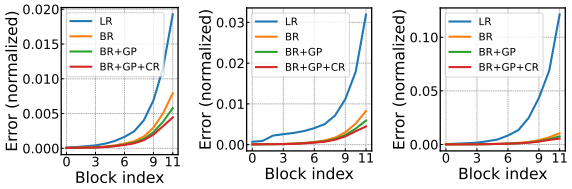
Example number	Text
1	*Americas, and the U.K., while 18 other countries have legalized the medical use of cannabis. The latest announcement is a win for Canadians ...*
2	*apprehension of the inevitability of death? And, therefore, how could such a person come to believe ...*
3	*'#' + this.currentID + .ิ\n };\n\n return {\n next: next,\n previous: previous,\n}...*
4	*Picker.setSelected(false);\n \n actionPhrasesTableModel.fireTableDataChanged();\n ...*

Table 5: Examples of self-generated data.



(a) SparseGPT (b) Wanda (c) Magnitude

Figure 5: Results of reconstruction techniques for LLaMA-7B. They constantly reduce the compounding errors, achieving a significant decrease at the final block ($87\% \sim 94\%$).



(a) SparseGPT (b) Wanda (c) Magnitude

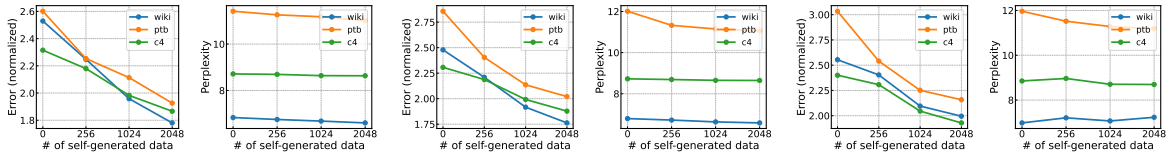
Figure 6: Results of reconstruction techniques for OPT-125M. They constantly reduce the compounding errors, achieving a significant decrease at the final block ($79\% \sim 96\%$).

Different techniques effectively improve the performance on perplexity and downstream tasks, with the exception of overfitting for CR on downstream tasks.

More results on self-generated data Reconstruction error on calibration data and test data for OPT-125M and LLaMA-7B are presented in Table 4. Decreased error for calibration data leads to decreased error for test data for OPT-125M, but leads to increased test error for LLaMA-7B.

Effects of self-generated calibration data are presented in Figure 7. In most cases, more number of self-generated data leads to decreased test error and perplexity.

Memory consumption of reconstruction techniques Solving (1) directly can be memory-



(a) SparseGPT (b) Wanda (c) Magnitude

Figure 7: Effects of self-generated calibration data on reconstruction error for test data and perplexity for LLaMA-7B; they both improve with more self-generation.

	LR	BR	BR+GP	BR+GP+CR	Full fine-tuning
peak memory (GB)	3.9	5.7	5.7	10.6	> 100

Table 6: Peak GPU memory for LLaMA-7B and sparseGPT. Compared to LR, reconstruction techniques incur additional GPU memory but it is quite marginal compared to fine-tuning the full model. The results are obtained with the batch size of 8 and gradient accumulation. For full fine-tuning, the results are from Malladi et al. (2023).

intensive, thus many recent work suggest divide-and-conquer such as LR and BR. In the work of Frantar and Alistarh (2023), the authors show that for the 175B parameter OPT model it requires at least five A100 GPUs of 80GB, whereas by using LR it reduces down to a single A100 GPU of 80GB. In our experiments, for Llama-7B, both LR and BR+GP+CR can all be done on a commodity 3090 GPU of 24GB memory; it requires more than 100GB to perform full fine-tuning of LLaMA-7B (Malladi et al., 2023). In theory, optimizing more parameters can incur more memory footprints, and thus, in the order of LR $=$ GP $<$ BR $<$ CR, there will be more memory usage.

The exact amount depends on the specific model. To provide solid evidence, we ran profiling peak GPU memory for LLaMA-7B with the batch size of 8 (see Table 6 for the results). Compared to LR, reconstruction techniques surely incur additional GPU memory, however, (i) it is quite marginal compared to fine-tuning the full model, and (ii) it could be reduced further by introducing memory reduction techniques in practice such as CPU offloading and gradient checkpointing.

Pruning attention vs. feed-forward We also investigated the effects of only pruning attention vs. feed-forward blocks for different reconstruction techniques. Here, we conducted experiments for OPT-125m and SparseGPT by pruning either attention or feed-forward blocks to 50% sparsity and measuring the perplexity on raw-Wikitext2. The results are provided in Table 7. We first observe that pruning both attention and feed-forward yields the largest performance drop. Also, we find that pruning only the attention block leads to worse performance compared to pruning only the feed-forward block, which is consistent with the findings in the previous work (Namburi et al., 2023). Interestingly, we find that reconstruction techniques can be more effective for cases with poor performance; *i.e.*, in the order of pruning all blocks > pruning attention > pruning feed-forward, BR, GP, CR reconstruction techniques yield more reduction in perplexity (which is good by itself).

C Details on Self-generation of Calibration Data

We generate additional calibration data from the original dense model. Here, we sample 10240 number of English texts each containing 2048 tokens. Specifically, we first randomly choose the initial token and generate four subsequent tokens by deterministically selecting top-1 predictions, similar to Liu et al. (2023). Here, we resample the tokens if the generated texts are not detected as English. Then, we stochastically generate the remaining tokens until the <EOS> token is produced or the sequence length exceeds 2048. Finally, the additional calibration data can be obtained by sampling a subset of generated texts and randomly selecting the intermediate 1024 tokens for each text.

Examples of self-generated texts are presented in Table 5. Examples 1 and 2 are plain English texts and can serve as good calibration data. However, we observe that programming codes such as examples 3 and 4 are often generated, which might not serve as good calibration data for improving the perplexity for English texts or accuracy for downstream tasks which are not related to code generation. In this regard, we believe that generating only

Pruning block	LR	BR	BR+GP	BR+GP+CR
Attention	32.82	30.15	29.97	29.64
Feed-forward	30.69	29.23	28.89	28.73
All	36.35	31.94	31.57	30.86

Table 7: Effects of pruning block for different reconstruction techniques. Here, we prune either attention or feed-forward block to 50% sparsity and measure the perplexity on raw-Wikitext2. Pruning only the attention block leads to worse performance compared to pruning only the feed-forward block. The results are for OPT-125m with sparseGPT.

a few number of high-quality texts can lead to improved performance while reducing computational costs.

Here, the generated data do not contain personal information or offensive content.